

Отчет к Лабораторной работе N°3 "Численное интегрирование"

Шуман Владислав Олегович

3 курс, 4 группа

Вариант 1.2

[Github](#)

Время последнего изменения: 03.05.2026

Часть из электронных конспектов, которую я использовал:

- [тут](#)
- [или тут \(Github\) в папке лаб3](#)

Отчет будет лежать в

- [папке отчет](#)
- [папке отчет, на Github](#)

Условие

с полным условием можно ознакомиться

- [тут](#)
- [или тут \(Github\)](#)

Задание 1

Рассмотрим интеграл $S(f) = \int_a^b f(x) dx$ и две составные квадратурные формулы (КФ), которые обозначим А и Б.

- Оценить N_f — количество вычислений функции f , которое нужно выполнить для вычисления интеграла с точностью 10^{-7} по формулам А и Б. *Занести ход вычислений и полученные результаты в отчет, не забудьте и сами квадратурные формулы.*
- Для числа разбиений отрезка $N = 10^{-k}$, $k = 1, \dots, 10$, вычислить приближенное значение интеграла по каждой из данных составных КФ и построить совмещенную логарифмическую диаграмму сходимости, по оси абсцисс отмечая количество вычислений N_f , по оси ординат — погрешность $|S(f) - Q_N(f)|$. Сравнить полученные экспериментальные результаты с теоретической оценкой, сделанной в предыдущем пункте, объяснить их. *Сделать соответствующие комментарии в отчете.*
- На основе любой из формул (А или Б) разработать алгоритм численного интегрирования по правилу Рунге с заданной точностью ϵ . При переходе к более частой сетке алгоритм должен рационально использовать информацию о ранее вычисленных значениях подынтегральной функции (если это возможно). Построить диаграммы сходимости, аналогичные предыдущему пункту. Применить построенный алгоритм с требуемой точностью $\epsilon = 10^{-k}$, $k = 1, \dots, 10$. Построить диаграмму сходимости, сравнить его эффективность с реализованными ранее алгоритмами без контроля точности.
- В отчет занести (помимо указанных выше результатов, диаграмм и программного кода):
 - Точное значение интеграла (если интеграл не берется, можно найти численно с точностью не менее 10^{-16} с помощью любой сторонней программы)
 - Описание алгоритма численного интегрирования,
 - Комментарии по результатам эксперимента.

2. КФ средних прямоугольников и трапеций, $f(x) = \frac{\sin(2x)}{1+x}$, $[a, b] = [1, 10]$

Решение

Рассмотрим интеграл $S(f) = \int_a^b f(x) dx$ и две составные квадратурные формулы (КФ), которые обозначим А и Б.

Пункт 1

- Оценить N_f — количество вычислений функции f , которое нужно выполнить для вычисления интеграла с точностью 10^{-7} по формулам А и Б. *Занести ход вычислений и полученные результаты в отчет, не забудьте и сами квадратурные формулы.*

составные КФ Средних прямоугольников

составную формулу средних прямоугольников (рис. 6.1):

$$\int_a^b f(x) dx \approx h \sum_{k=1}^N f\left(a + \left(k - \frac{1}{2}\right)h\right). \quad (6.23)$$

составные КФ Трапеций

Готово. Итак, составная КФ трапеций имеет вид

$$\int_a^b f(x) dx \approx \frac{h}{2} \left(f(a) + 2 \sum_{k=1}^{N-1} f(a + kh) + f(b) \right). \quad (15.9)$$

Следовательно, формула для остатка составной КФ примет вид

$$R_n^N(f) = Ch^l N f^{(m)}(\eta), \quad (15.8a)$$

или, с учётом $h = (b - a)/N$,

$$R_n^N(f) = C \frac{(b - a)^l}{N^{l-1}} f^{(m)}(\eta), \quad (15.8b)$$

или, с учётом $N = (b - a)/h$,

$$R_n^N(f) = C(b - a)h^{l-1} f^{(m)}(\eta). \quad (15.8c)$$

Здесь $\eta \in [a, b]$. На практике предпочтительнее использовать последние две формы остатка.

как получить постоянные из формулы

Теперь построим формулу для остатка согласно (15.8). Для этого вспомним, что простая формула трапеций имеет остаток вида

$$R_1(f) = -\frac{(b - a)^3}{12} f''(\eta),$$

то есть $C = -1/12$, $l = 3$, $m = 2$. Тогда из (15.8c) получаем

$$R_1^N(f) = -\frac{b - a}{12} h^2 f''(\eta), \quad \eta \in [a, b]. \quad (15.10)$$

остаток для средних прямоугольников для простейших интерполяционных КФ

$$R_0(f) = f''(\eta) \frac{(b - a)^3}{24}. \quad (13.13)$$

..

$$N \geq \left(\frac{C(b - a)^l}{\varepsilon} \|f^{(m)}\|_{C[a,b]} \right)^{\frac{1}{l-1}} \Rightarrow |R_n^N| \leq \varepsilon.$$

Лаб 3

даны N — кон. количество
 f , к-е. нулю вкл. для вкл.
 интервалов $\Delta x = 10^{-4}$ по
 формулам A и B

даны a, b , разбито на N
 частей $h = \frac{b-a}{N}$, узлами
 $x_k = a + kh, k = 0, N$

сов. кр. средних прямоугол.:

$$Q_N^A(f) = h \sum_{k=1}^N f(a + (k - \frac{1}{2})h)$$

сов. кр. трапеций:

$$Q_N^B(f) = \frac{h}{2} (f(a) + 2 \sum_{k=1}^{N-1} f(a + kh) + f(b))$$

другой вид остатка сов. кр.:

$$R_N^m(f) = C \frac{(b-a)^p}{N^{p-1}} f^{(m)}(\xi)$$

где C, m, p — const, ξ — точка из отрезка $[a, b]$.

даны
 нулю

году не менее
 $|R_N^m(f)| \leq \varepsilon$

точность ε

$$\text{i.e. } \left| C \frac{(b-a)^p}{N^{p-1}} f^{(m)}(\eta) \right| \leq \varepsilon$$

$$\frac{|C| (b-a)^p}{N^{p-1}} \|f^{(m)}\|_{C[a,b]} \leq \varepsilon$$

$$\text{also: } N \geq \left(\frac{|C| (b-a)^p}{\varepsilon} \max_{x \in [a,b]} |f^{(m)}(x)| \right)^{1/p-1}$$

А:
 где простейший: $R_0(x) = \frac{(b-a)^3}{24} f''(\eta) \Rightarrow$

$$\Rightarrow C = \frac{1}{24}, p=3, m=2$$

где B: $C = -1/12, p=3, m=2$

Всего $\max_{[a,b]} |f^{(2)}(x)|$

$$f(x) = \frac{\sin 2x}{1+x}$$

$$f'(x) = \frac{2 \cos(2x) (1+x) - \sin 2x}{(1+x)^2}$$

$$= \frac{2 \cos 2x}{1+x} - \frac{\sin(2x)}{(1+x)^2}$$

$$f''(x) = \frac{-4 \sin(2x)(1+x) - 2\cos(2x)}{(1+x)^2} -$$

$$\frac{2\cos(2x)(1+x)^2 - 2\sin(2x)(x+1)}{(1+x)^4}$$

исполн

$$f''(x) = \frac{-4 \sin(2x)}{1+x} - \frac{4 \cos(2x)}{(1+x)^2} + \frac{2 \sin(2x)}{(1+x)^3}$$

$$|f''(x)| \leq \frac{4}{1+x} + \frac{4}{(1+x)^2} + \frac{2}{(1+x)^3} -$$

монотонно убывает $\Rightarrow \max_{x \in [0,1]}$

$$\Rightarrow \frac{4}{1+1} + \frac{4}{(1+1)^2} + \frac{2}{(1+1)^3} = \frac{4}{2} + \frac{4}{4} + \frac{2}{8} =$$

$$= 3,25$$

исполн:

$$N_A \geq \sqrt{\frac{\frac{1}{24} \cdot 9^3}{10^{-4}}} \cdot 3,25 \approx 31419,54$$

$$N_A = 31420.$$

$$N_B \geq \sqrt{\frac{\frac{1}{12} \cdot 9^3}{10^{-4}}} \cdot 3,25 \approx 44433,95$$

$$N_B = 44434, + 1 =$$

$$= 44435.$$

БАНК

Итого $N_A = 31420$ и $N_B = 44435$

Пункт 2

- Для числа разбиений отрезка $N = 10^{-k}$, $k = 1, \dots, 10$, вычислить приближенное значение интеграла по каждой из данных составных КФ и построить совмещенную логарифмическую диаграмму сходимости, по оси абсцисс отмечая количество вычислений N_f , по оси ординат — погрешность $|S(f) - Q_N(f)|$. Сравнить полученные экспериментальные результаты с теоретической оценкой, сделанной в предыдущем пункте, объяснить их. *Сделать соответствующие комментарии в отчете.*

$$[a, b] = [1, 10]$$

с точностью 10^{-7}

```
# входные параметры
a, b = 1.0, 10.0 # отрезок [a,b]
eps = 1e-7 # погрешность
```

$$f(x) = \frac{\sin(2x)}{1+x},$$

```
import numpy as np

# функция f
def f(x):
    return np.sin(2 * x) / (1 + x)
```

- Точное значение интеграла (если интеграл не берется, можно найти численно с точностью не менее 10^{-16} с помощью любой сторонней программы)


```
# "эталонное" значение,
from scipy.integrate import quad

S_f, _ = quad(f, a, b, epsabs=1e-16)
```

составную формулу средних прямоугольников (рис. 6.1):

$$\int_a^b f(x)dx \approx h \sum_{k=1}^N f\left(a + \left(k - \frac{1}{2}\right)h\right). \quad (6.23)$$

```
def composite_formula_of_medium_rectangles(a, b, N, chunk_size=10**7):
    h = (b - a) / N # шаг
    total_sum = 0.0 # тут копить будем
    for k in range(1, N + 1, chunk_size):
        current_n = min(chunk_size, N + 1 - k) # либо размер шага,
        # либо если он меньше, то N - i
        indices = np.arange(k, k + current_n) # индексы на текущей
        # итерации
        x_mid = a + (indices - 0.5) * h # иксы на текущей итерации
        total_sum += np.sum(f(x_mid)) # добавить часть суммы для
        # текущей итерации
    return h * total_sum, N
```

Готово. Итак, составная КФ трапеций имеет вид

$$\int_a^b f(x)dx \approx \frac{h}{2} \left(f(a) + 2 \sum_{k=1}^{N-1} f(a + kh) + f(b) \right). \quad (15.9)$$

```
def composite_formula_of_trapezoids(a, b, N, chunk_size=10**7):
    h = (b - a) / N
    internal_points_sum = 0.0

    # 2 * СУММА(...)
    for k in range(1, N, chunk_size):
        current_n = min(chunk_size, N - k)
        indices = np.arange(k, k + current_n)
        x_k = a + indices * h
        internal_points_sum += np.sum(f(x_k))

    # ИТОГ
    integral = (h / 2) * (f(a) + f(b) + 2 * internal_points_sum)
    return integral, N + 1
```

Для числа разбиений отрезка $N = 10^{-k}$, $k = 1, \dots, 10$, вычислить приближенное значение интеграла по каждой из данных составных КФ и построить совмещенную логарифмическую диаграмму сходимости, по оси абсцисс отмечая количество вычислений N_f , по оси ординат — погрешность

```
import time

k_values = np.arange(1, 11) # k 1 -- 10
results = [] # результаты

print(f"{'k':<3} | {'N':<11} | {'метод':<15} | {'погрешность':<12} |  
{ 'время (в секундах)':<10}")
print("-" * 65)

for k in k_values:
    N = 10**k

    # средние прямоугольники
    start_medium_rectangles = time.time()
    val_medium_rectangles, Nf_medium_rectangles =  
composite_formula_of_medium_rectangles(a, b, N)
    error_medium_rectangles = abs(S_f - val_medium_rectangles)
    t_medium_rectangles = time.time() - start_medium_rectangles

    # трапеции
    start_trapezoids = time.time()
    val_trapezoids, Nf_trapezoids = composite_formula_of_trapezoids(a,  
b, N)
    error_trapezoids = abs(S_f - val_trapezoids)
    t_trapezoids = time.time() - start_trapezoids

    results.append((Nf_medium_rectangles, error_medium_rectangles,  
Nf_trapezoids, error_trapezoids))

    print(f"{'k':<3} | 10^{'k':<8} | прямоугольники |  
{error_medium_rectangles:<12.2e} | {t_medium_rectangles:<10.2f}")
    print(f"{'k':<3} | {'N':<11} | трапеции |  
{error_trapezoids:<12.2e} | {t_trapezoids:<10.2f}")
    print("-" * 65)
```

k	eps	N итоговое	N предск.	N_f (всего)	оценка ошибки	ошибка
1	10^{-1}	4	3	5	$3.06e-02$	$4.96e-01$
2	10^{-2}	32	23	33	$4.84e-03$	$4.71e-03$
3	10^{-3}	128	70	129	$2.93e-04$	$2.93e-04$
4	10^{-4}	256	219	257	$7.32e-05$	$7.31e-05$
5	10^{-5}	1024	693	1025	$4.57e-06$	$4.57e-06$
6	10^{-6}	4096	2190	4097	$2.86e-07$	$2.86e-07$
7	10^{-7}	8192	6924	8193	$7.14e-08$	$7.14e-08$
8	10^{-8}	32768	21894	32769	$4.46e-09$	$4.46e-09$
9	10^{-9}	131072	69234	131073	$2.79e-10$	$2.79e-10$
10	10^{-10}	262144	218937	262145	$6.98e-11$	$6.98e-11$

```
import matplotlib.pyplot as plt

# results.append((Nf_medium_rectangles, error_medium_rectangles,
# Nf_trapezoids, error_trapezoids))
(Nf_medium_rectangles_all,
 error_medium_rectangles_all,
 Nf_trapezoids_all,
 error_trapezoids_all) = zip(*results)

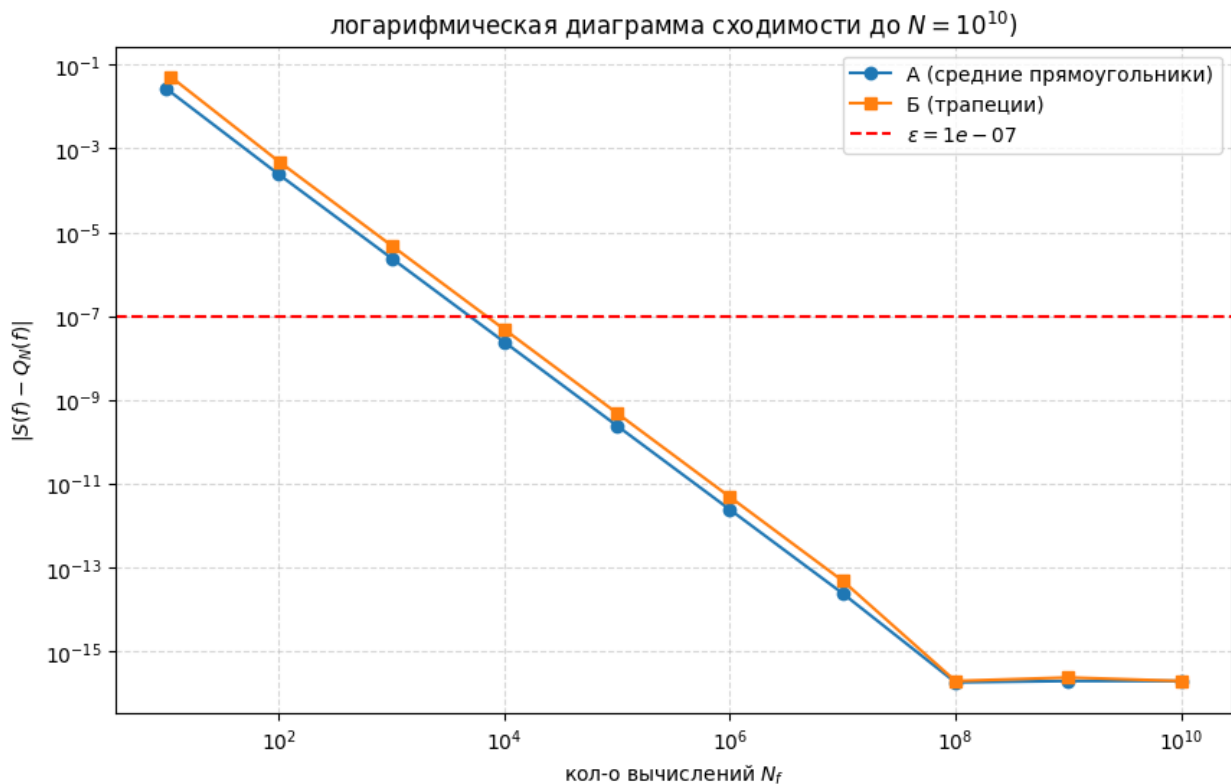
plt.figure(figsize=(10, 6))

plt.loglog(Nf_medium_rectangles_all, error_medium_rectangles_all,
'o-',
            label='А (средние прямоугольники)')
plt.loglog(Nf_trapezoids_all, error_trapezoids_all, 's-',
            label='Б (трапеции)')

plt.axhline(y=eps, color='r', linestyle='--', label=f'$\epsilon = \{eps\}$')

plt.title('логарифмическая диаграмма сходимости до $N=10^{\{10\}}$')
plt.xlabel('кол-о вычислений $N_f$')
plt.ylabel('$|S(f) - Q_N(f)|$')
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.legend()
plt.show()

<>:17: SyntaxWarning: invalid escape sequence '\e'
<>:17: SyntaxWarning: invalid escape sequence '\e'
/var/folders/24/jr600lf178n120v25n50gtvh0000gn/T/ipykernel_88064/23282
58788.py:17: SyntaxWarning: invalid escape sequence '\e'
    plt.axhline(y=eps, color='r', linestyle='--', label=f'$\epsilon = \{eps\}$')
```

$|S(f) - Q_N(f)|$. Сравнить полученные экспериментальные результаты с теоретической оценкой, сделанной в предыдущем пункте, объяснить их. *Сделать соответствующие комментарии в отчете.*

исходя из получившейся таблицы И графика, видно что погрешность у Прямоугольников в примерно 2 раза меньше, что похоже и на теоретический результат (константа $C_{прямоуг} = \frac{1}{24}$ в два раза меньше чем $C_{трап} = \frac{1}{12}$)

так же, теоретически у нас получилось $N_A = 31420$ и $N_B = 44435$, а практически $N_{практически} < 10000$, то есть результаты отличаются в 3-4 раза. связано это с тем, что теоретическая оценка, которую я использовал, является априорной и использует максимум производной на отрезке

$$N \geq \left(\frac{C(b-a)^l}{\varepsilon} \|f^{(m)}\|_{C[a,b]} \right)^{\frac{1}{l-1}} \Rightarrow |R_n^N| \leq \varepsilon.$$

Отметим, что данная оценка является априорной. Этот способ имеет два очевидных недостатка. Во-первых, вычисление $f^{(m)}$ часто является проблематичным, не говоря уже о нахождении нормы этой функции. Во-вторых, даже если $\|f^{(m)}\|_{C[a,b]}$ известна, то оценка такого вида может быть сильно завышенной.

Поэтому чаще применяют другой, апостериорный, способ оценки погрешности, называемый *правилом Рунге*, или *методом двойного пересчёта*.

начиная с $k = 8$ погрешность как бы достигает "низа" (так как мы используем float64, то там $\varepsilon \approx 2.22 \cdot 10^{-16}$)

Пункт 3

- На основе любой из формул (А или Б) разработать алгоритм численного интегрирования по правилу Рунге с заданной точностью ε . При переходе к более частой сетке алгоритм должен рационально использовать информацию о ранее вычисленных значениях подынтегральной функции (если это возможно). Построить диаграммы сходимости, аналогичные предыдущему пункту. Применить построенный алгоритм с требуемой точностью $\varepsilon = 10^{-k}$, $k = 1, \dots, 10$. Построить диаграмму сходимости, сравнить его эффективность с реализованными ранее алгоритмами без контроля точности.

алгоритм из теории

Практические рекомендации. Теперь поговорим о том, как всё это реализуется на практике. Простейший алгоритм выглядит следующим образом.

1. Выбираем базовую квадратурную сумму $Q_n = Q$, а также N_1 и N_2 , точность ε .
2. $q_1 \leftarrow Q^{N_1}(f)$, $q_2 \leftarrow Q^{N_2}(f)$.
3. По формуле (15.12) вычисляем оценку погрешности для «более точного» приближения q_2 :

$$\tilde{R} \leftarrow \frac{(q_2 - q_1)h_2^p}{h_1^p - h_2^p}.$$

Как мы помним, здесь p — показатель точности КФ, определяемый (15.11), $h_i = (b - a)/N_i$.

4. Если $|\tilde{R}| \leq \varepsilon$, полагаем $q \leftarrow q_2$ и завершаем алгоритм. В противном случае переходим к следующему шагу.
5. Вычисляем «оптимальное» количество разбиений N^* по формуле (15.14):

$$N^* \leftarrow \left[\left(\frac{|\tilde{R}|}{\varepsilon} \right)^{1/p} N_2 \right] + 1.$$

Здесь $[\cdot]$ — целая часть числа, а плюс единица — для надёжности.

6. Полагаем $N_1 \leftarrow N_2$, $q_1 \leftarrow q_2$, $N_2 \leftarrow N^*$, $q_2 \leftarrow Q^{N^*}(f)$.
7. Переходим к шагу 3.

так как я могу выбрать любую формулу, то возьму Б(составную КФ трапеций)

чтобы рационально воспользоваться ранее вычисленными значениями, воспользуюсь следующим советом

- **Экономия вычислений** f : в зависимости от используемой базовой КФ подбирать N_1 и N_2 так, чтобы в соответствующих сетках для вычисления q_1 и q_2 было как можно больше одинаковых узлов. Например, в случае составной формулы трапеций (15.9), обычно берут $N_{i+1} = 2N_i$. Тогда получается

$$Q_1^N(f) = \frac{h}{2} \left(f(a) + 2 \sum_{k=1}^{N-1} f(a + kh) + f(b) \right),$$

$$Q_1^{2N}(f) = \frac{h}{4} \left(f(a) + 2 \sum_{k=1}^{2N-1} f(a + k\frac{h}{2}) + f(b) \right) =$$

$$= \frac{1}{2} \left(Q_1^N(f) + h \sum_{k=1}^N f(a + (2k-1)\frac{h}{2}) \right).$$

$$\bar{R} = \frac{5h_2^p}{h_1^p - h_2^p}$$

$$p=2, \quad h_1 = 2h_2 \quad S_2 = 2S_1$$

$$\bar{R} = \frac{(2S_2 - S_1) h_2^2}{4h_2^2 - h_2^p} = \frac{2S_2 - S_1}{3}$$

```
def runge_trapezoidal(a, b, target_precision):
    N = 1
    h = (b - a) / N
    integral_prev = (h / 2) * (f(a) + f(b)) # для первой итерации, N =
1
    evaluations_count = 2 # счетчик. сколько функций посчитано. чтобы
    вернуть N
```



```

runge_error = float('inf') # хранить ошибку Рунге

while runge_error > target_precision:
    N *= 2 # удваиваем
    h_new = (b - a) / N # h*

    # у нас остались старые узлы при N
    # надо посчитать новые при N* = 2N
    # а они расположены по a + (2 * 1 - 1)h_new, a + (2 * 2 -
1)*h_new, a + (2 * 3 - 1)*h_new ...
    indices = np.arange(1, N, 2)
    x_new = a + indices * h_new

    sum_for_new = np.sum(f(x_new)) # сумма для новых
    evaluations_count += len(x_new) # счетчик прибавляем

    # пересчитываем по формуле
    integral_current = 0.5 * integral_prev + h_new * sum_for_new

    runge_error = abs(integral_current - integral_prev) / 3.0

    integral_prev = integral_current

    N_predicted = int(np.ceil((runge_error / target_precision)**(1/2)
* N))
    return integral_current, evaluations_count, runge_error, N,
N_predicted

results_runge = []

print(f"{'k':<3} | {'eps':<9} | {'N итоговое':<10} | {'N предск.':<10}
| {'N_f (всего)':<11} | {'оценка ошибки':<12} | {'ошибка':<12}")
print("-" * 85)

for k in k_values:
    target_eps = 10.0**(-k) # eps для итерации
    val_runge, Nf_runge, err_runge_est, final_N, predicted_N =
runge_trapezoidal(a, b, target_eps) # функцию применяем
    error = abs(S_f - val_runge) # ошибка
    results_runge.append((Nf_runge, error, target_eps)) # пополняем
результаты
    print(f"{'k':<3} | 10^{'-k':<7} | {'final_N':<10} | {'predicted_N':<10} |
{'Nf_runge':<11} | {'err_runge_est':<12.2e} | {'error':<12.2e}")

```

k	eps	N итоговое	N предск.	N_f (всего)	оценка ошибки	ошибка
1	10^{-1}	4	3	5	$3.06e-02$	$4.96e-01$
2	10^{-2}	32	23	33	$4.84e-03$	$4.71e-03$
3	10^{-3}	128	70	129	$2.93e-04$	$2.93e-04$
4	10^{-4}	256	219	257	$7.32e-05$	$7.31e-05$
5	10^{-5}	1024	693	1025	$4.57e-06$	$4.57e-06$
6	10^{-6}	4096	2190	4097	$2.86e-07$	$2.86e-07$
7	10^{-7}	8192	6924	8193	$7.14e-08$	$7.14e-08$
8	10^{-8}	32768	21894	32769	$4.46e-09$	$4.46e-09$
9	10^{-9}	131072	69234	131073	$2.79e-10$	$2.79e-10$
10	10^{-10}	262144	218937	262145	$6.98e-11$	$6.98e-11$

Алгоритм До, просто теперь он не делает range(1, N), а массив N, которые мы хотим

```
def composite_formula_of_trapezoids(a, b, N_input, chunk_size=10**7):
    # если пришел список
    if isinstance(N_input, (list, np.ndarray)):
        results_list = []
        for n_val in N_input:
            val, nf = composite_formula_of_trapezoids(a, b, n_val,
chunk_size)
            results_list.append((val, nf))
        return results_list

    # для одного N
    N = int(N_input)
    h = (b - a) / N
    internal_points_sum = 0.0
    for k_start in range(1, N, chunk_size):
        current_n = min(chunk_size, N - k_start)
        indices = np.arange(k_start, k_start + current_n)
        x_k = a + indices * h
        internal_points_sum += np.sum(f(x_k))

    integral_value = (h / 2) * (f(a) + f(b) + 2 * internal_points_sum)
    return integral_value, N + 1

N_values_from_runge_algorithm = [(item[0] - 1) for item in
results_runge] # список N из алгоритма для Рунге

trapezoids_for_runge_values = composite_formula_of_trapezoids(a, b,
N_values_from_runge_algorithm) # просчитываем для предыдущего
алгоритма
values_trapezoids, nf_trapezoids = zip(*trapezoids_for_runge_values)
errors_trapezoids = [abs(S_f - value) for value in values_trapezoids]
```

```

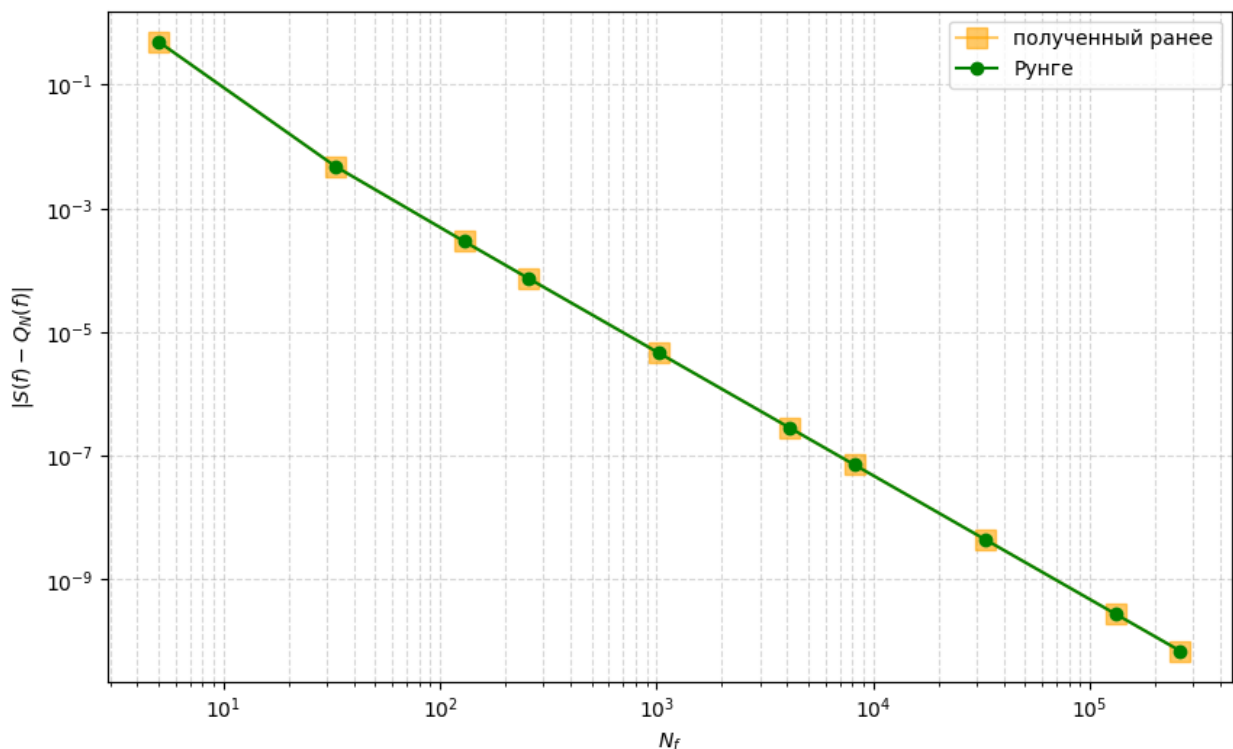
nf_runge_all, error_runge_all, target_eps_all = zip(*results_runge) #
для Рунге

# График
plt.figure(figsize=(10, 6))
plt.loglog(nf_trapezoids, errors_trapezoids, 's-',
           color='orange', alpha=0.6, label='полученный ранее',
           markersize=10)

# Результаты самого алгоритма Рунге
plt.loglog(nf_runge_all, error_runge_all, 'o-',
           color='green', label='Рунге', markersize=6)

plt.xlabel('$N_f$')
plt.ylabel('$|S(f) - Q_N(f)|$')
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.legend()
plt.show()

```



- графики идентичны. связано с тем, что мы используем одну и ту же формулу трапеций. а основная разница в том, что -- в без-контроля-остановки мы задаем N, а в Рунге -- точность, которую хотим получить
- $N_{теор} = 44435$ и $N_{с-контролем-точности} = 8193$, что 5.4 раза меньше. то есть -- оценка получилось сильно завышенной
- сравнивая их эффективность

- в алгоритме-без-контроля-точности, чтобы достичь нужной точности, мы должны
 - либо самим оценить N . Но тогда, если бы мы воспользовались нашим теоретическим предположением, то мы бы сделали в 5.4 раза больше вычислений, чем было достаточно
 - либо итерировались по значениям N_i для достижения нужной точности, а это порядка $\sum N_i$
 - а в алгоритме с контролем точности мы останавливаемся на том моменте, когда достигли нужной точности и так же эффективно переиспользуем узлы, чтобы не делать лишних вычислений
 - но так же может быть случай, когда $N_{\text{оптимальное}}$ лежит между N и $2N$, из-за чего мы можем сделать так же лишние вычисления
- В отчет занести (помимо указанных выше результатов, диаграмм и программного кода):
- Точное значение интеграла (если интеграл не берется, можно найти численно с точностью не менее 10^{-16} с помощью любой сторонней программы)
 - Описание алгоритма численного интегрирования,
 - Комментарии по результатам эксперимента.

```
print(S_f)
-0.07054363235342234
```